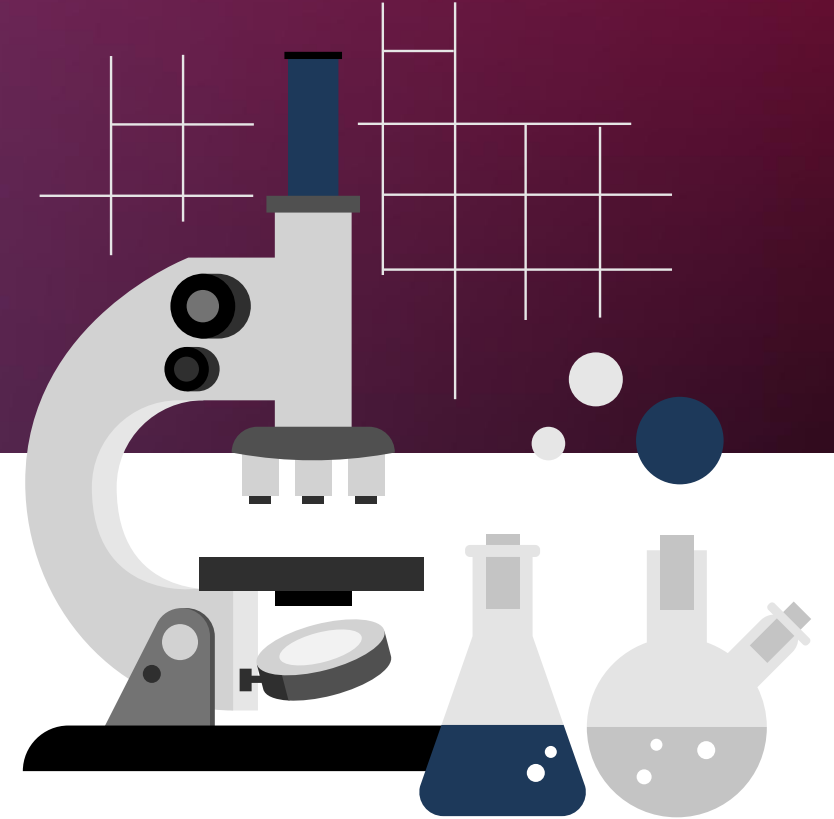


Cours rédigé par :

Mathieu SEURIN

mathieu.seurin@devinci.fr

ANALYSE DE DONNÉES



RAPPEL NOTATION JEU DE DONNÉES

	x_0	x_1
	Taille en m2	Terrasse
$x^{(0)}$	25	0
	100	0
	110	0
	175	1
$x^{(5)}$	200	1

X

Prix
275k
400k
500k
700k
750k

$y^{(0)}$

$y^{(n)}$

y

$$n = 5$$

Nombre d'exemples

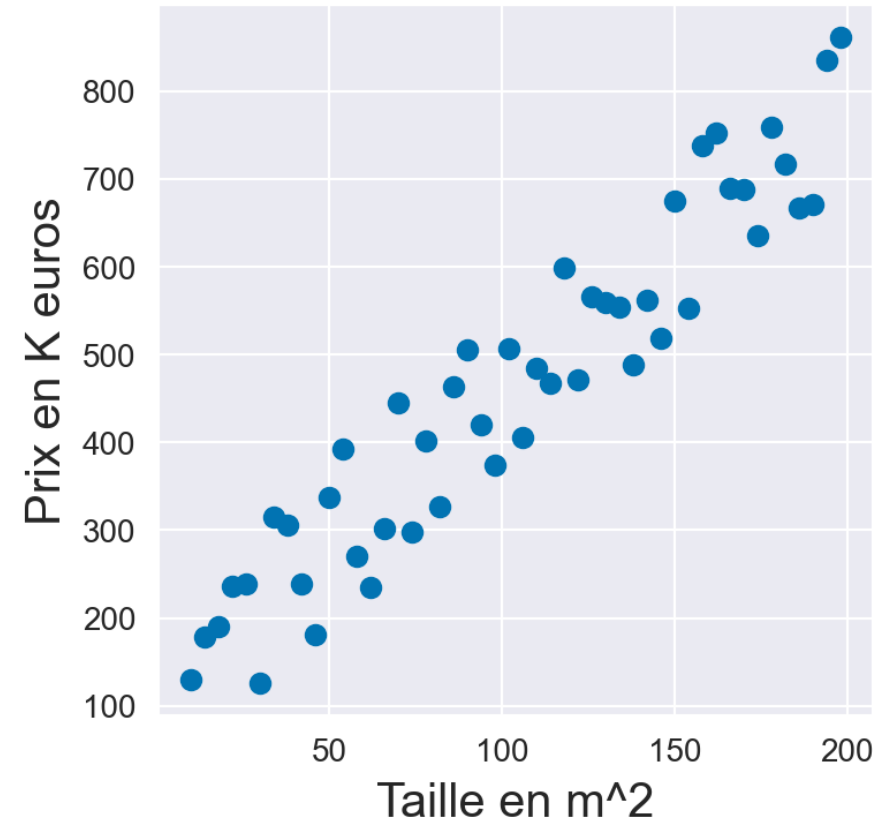
$$k = 2$$

Nombre de caractéristiques

N'oubliez pas la séparation train/test

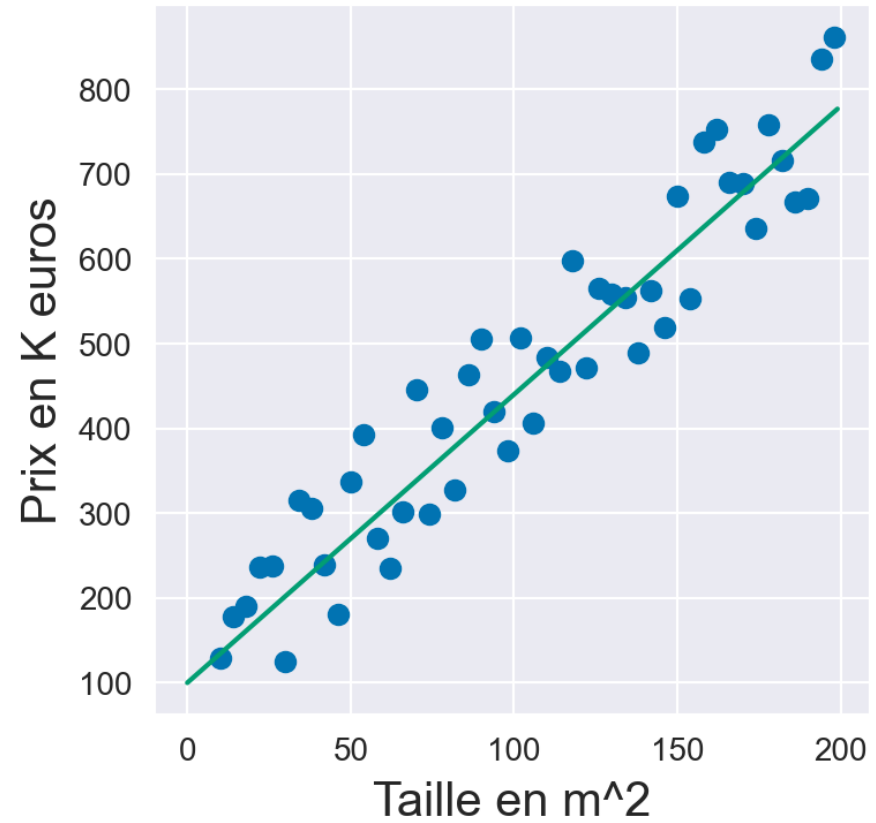
PRÉDICTION AVEC 1 CARACTÉRISTIQUE

x_0	
$x^{(0)}$	Taille en m2
	25
	100
	110
	175
$x^{(n)}$	200
X	
Prix	
$y^{(0)}$	
275k	
400k	
500k	
700k	
$y^{(n)}$	
750k	
y	



OBJECTIF

On cherche : $\theta = [\theta_0, \theta_1]$
avec : $\hat{y} = \theta_0 + \theta_1 x$

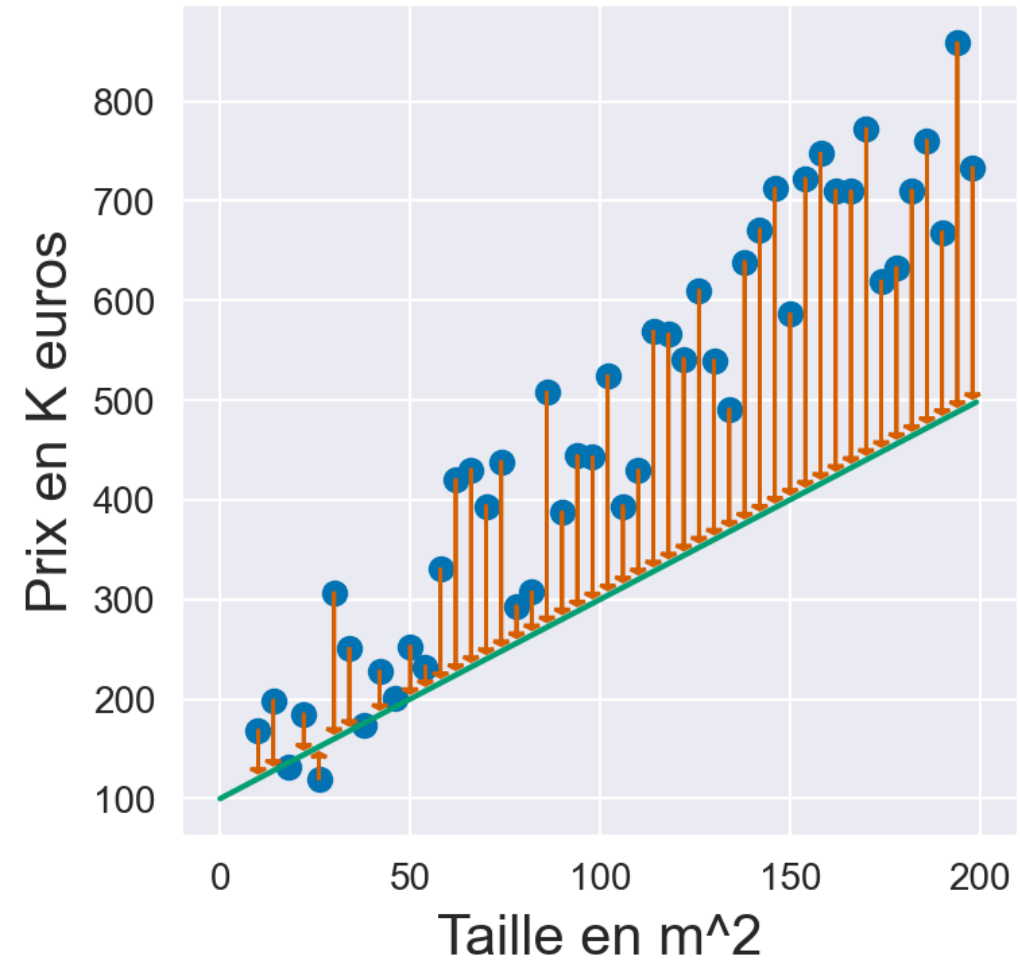


OBJECTIF

On cherche : $\theta = [\theta_0, \theta_1]$
avec : $\hat{y} = \theta_0 + \theta_1 x = f_\theta(x)$

Fonction de coût :

$$\sum_{i=0}^n \|y^{(i)} - f_\theta(x^{(i)})\|^2$$



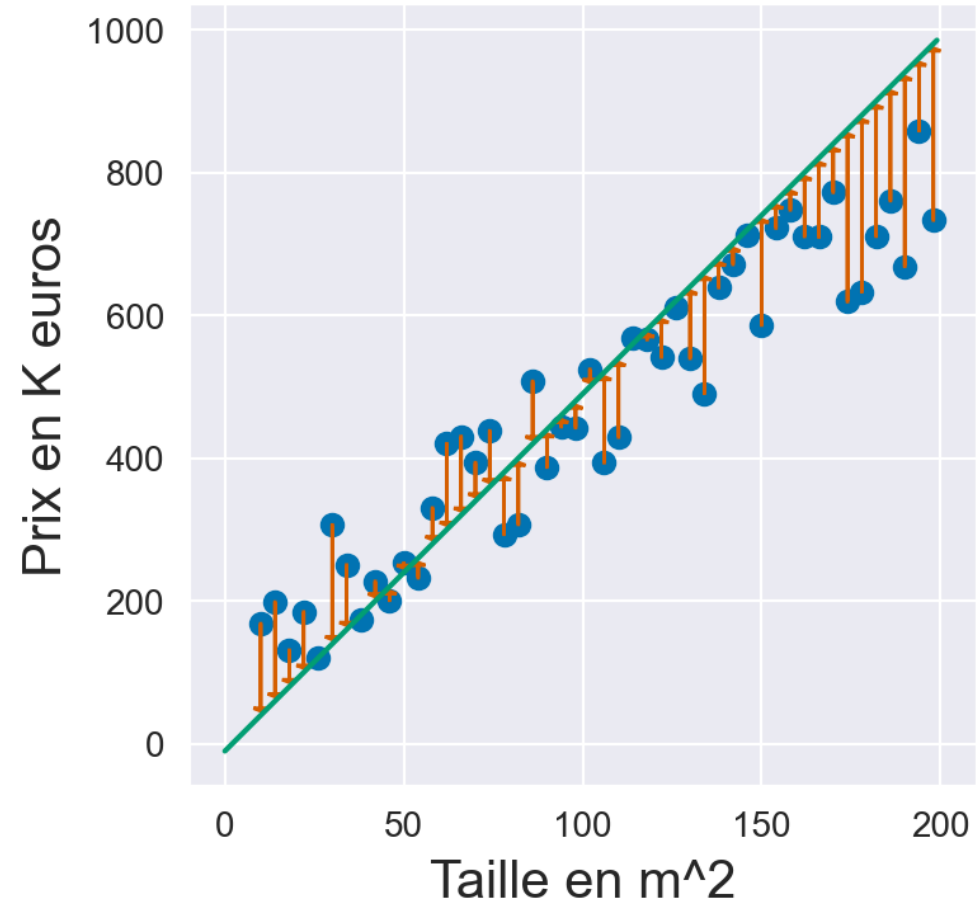
OBJECTIF

On cherche : $\theta = [\theta_0, \theta_1]$
avec : $\hat{y} = \theta_0 + \theta_1 x = f_\theta(x)$

Fonction de coût :

$$\sum_{i=0}^n \|y^{(i)} - f_\theta(x^{(i)})\|^2$$

Cette fonction s'appelle les
moindres carrés
(MSE = Mean Square Error)



OBJECTIF

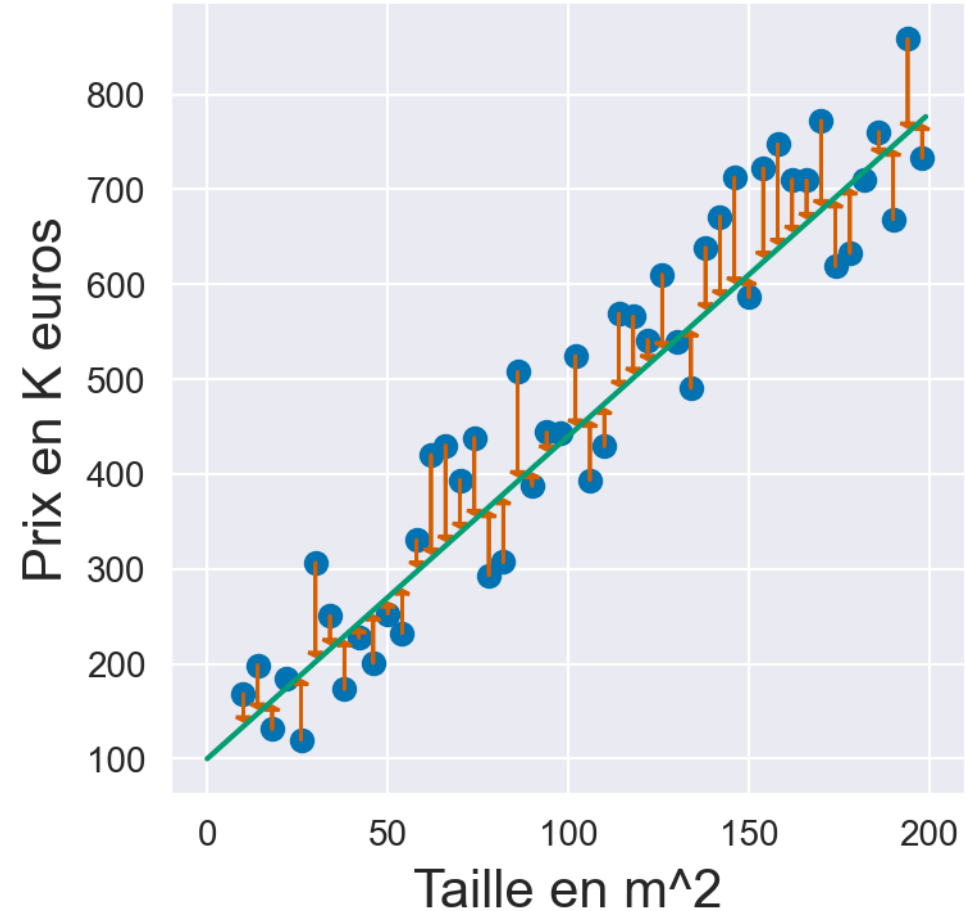
On cherche : $\theta = [\theta_0, \theta_1]$
avec : $\hat{y} = \theta_0 + \theta_1 x = f_{\theta}(x)$

Moindres Carrés (MSE)

$$\sum_{i=0}^n \|y^{(i)} - f_{\theta}(x^{(i)})\|^2$$

Objectif trouver θ tel que :

$$\min_{\theta} \sum_{i=0}^n \|y^{(i)} - f_{\theta}(x^{(i)})\|^2$$



QUELLES PROPRIÉTÉS DU COUT ?

Formulation générale de la tâche de régression :

$$J(\theta) = \sum_{i=0}^n \|y^{(i)} - f_{\theta}(x^{(i)})\|^2$$

Formulation pour la régression linéaire 1D

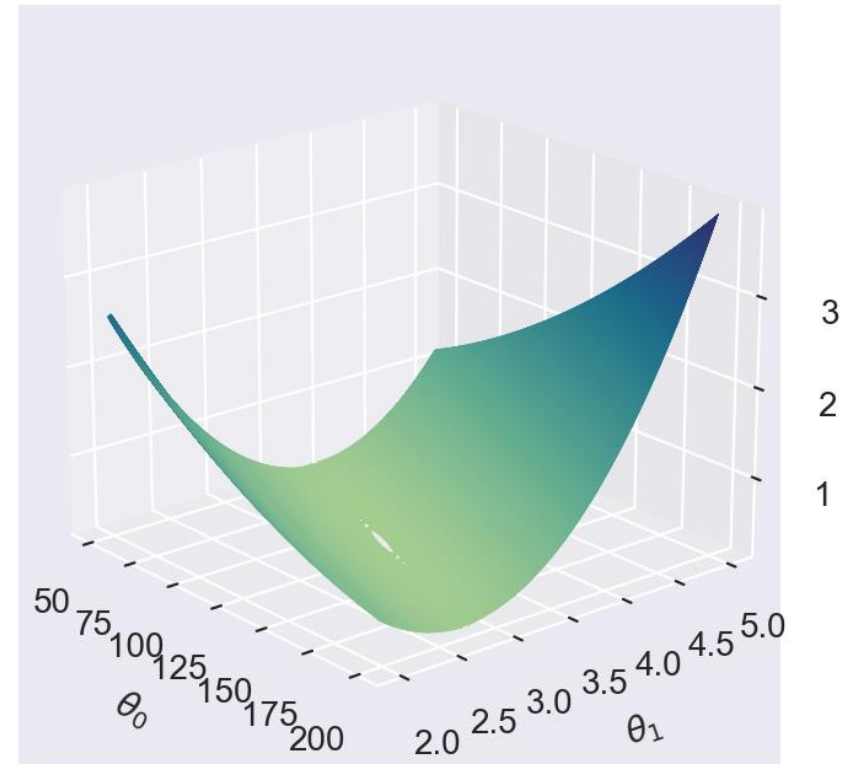
$$J(\theta_0, \theta_1) = \sum_{i=0}^n \|y^{(i)} - (\theta_0 + \theta_1 x)\|^2$$

Propriété : Dans le cadre de la régression linéaire, la fonction J est une fonction convexe.

(Quelle que soit le nombre de caractéristiques et donc de paramètres θ)

Par contre, si je change f_{θ} , la propriété ne tient plus

MSE en fonction de θ_0 et θ_1



QUELLES PROPRIÉTÉS DU COUT ?

Formulation générale de la tâche de régression :

$$J(\boldsymbol{\theta}) = \frac{1}{2} \|X\boldsymbol{\theta} - y\|^2$$

$$\nabla J(\boldsymbol{\theta}) = X^T(X\boldsymbol{\theta} - y),$$

$$\nabla^2 J(\boldsymbol{\theta}) = X^T X$$

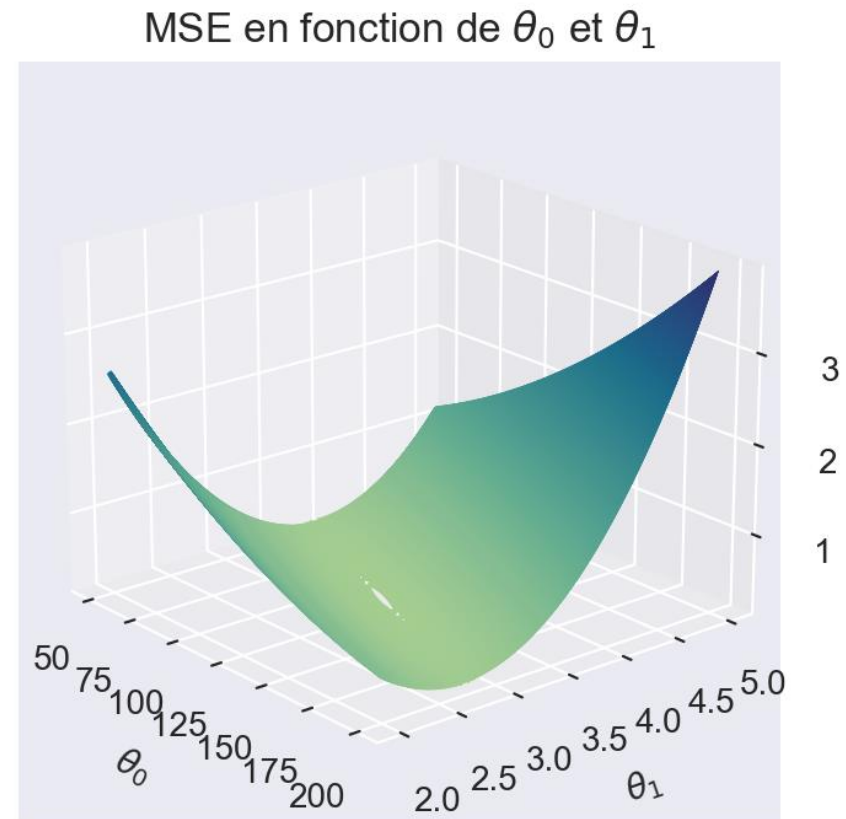
$X^T X$ étant semi-définie positive, $J(\boldsymbol{\theta})$ est convexe

Qu'est ce que cela veut dire pour nous ?

Il existe un seul minimum local, qui est un minimum global.

En Machine Learning, cela veut dire :

Il existe un $\boldsymbol{\theta}$ tel que mon coût est minimal et donc que ma régression linéaire est au plus proche des points (compte tenu des caractéristiques que l'on a)



COMMENT TROUVER CE MINIMUM ?

Solution analytique

$$\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \min_{\boldsymbol{\theta}} ||X\boldsymbol{\theta} - y||^2$$

Minimum local quand la dérivée s'annule (et grâce à la convexité, c'est un minimum global)

$$\nabla J(\boldsymbol{\theta}) = 0$$

$$X^T(X\boldsymbol{\theta} - y) = 0$$

$$\boldsymbol{\theta} = (X^T X)^{-1} X y$$

Problème de l'inversion de matrice :

Si il y a beaucoup de caractéristiques, va prendre beaucoup de temps

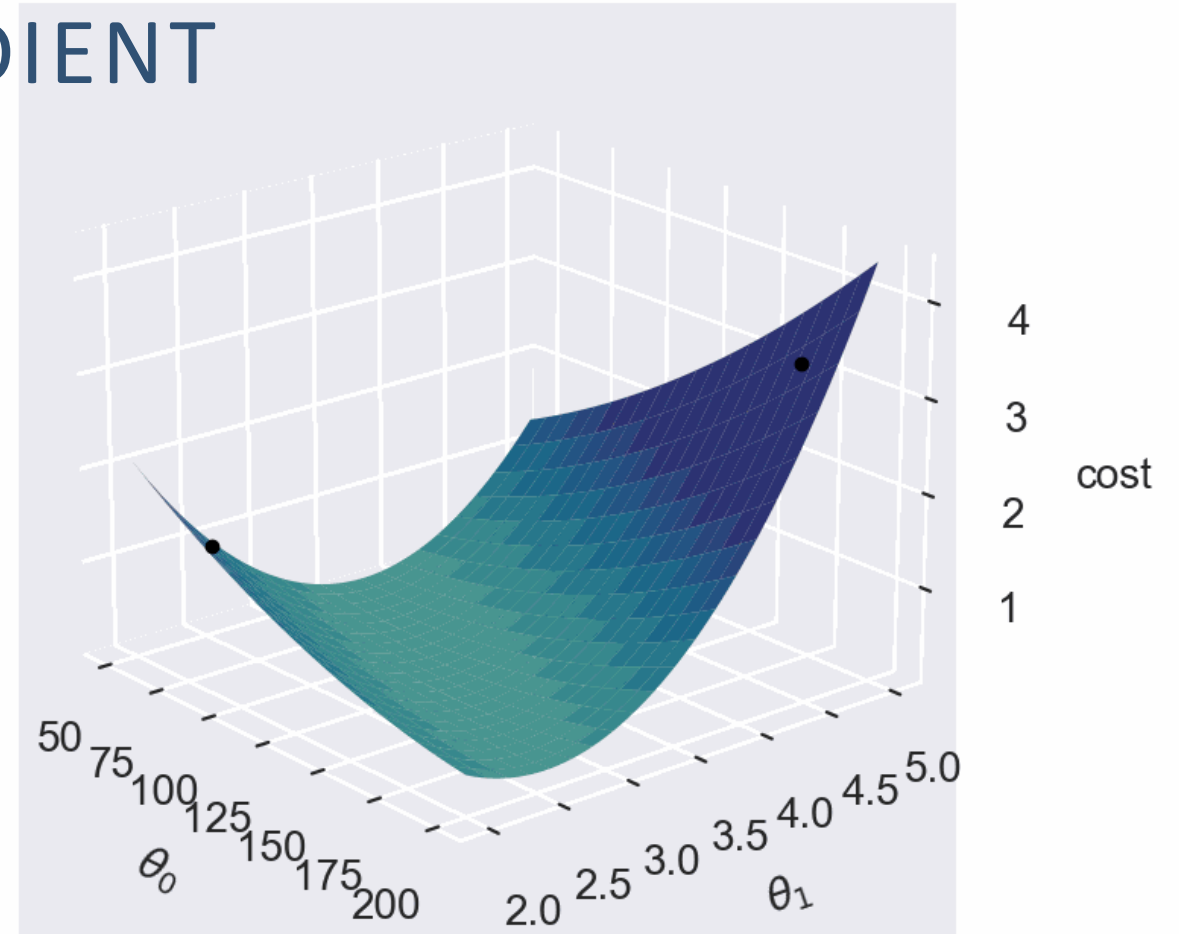
Dataset avec 10000 caractéristiques : ~ 1min

AUTRE POSSIBILITÉ : DESCENTE DE GRADIENT

Algorithme itératif qui permet de trouver un minimum local (dans sa version de base)

Utilisé dans les réseaux de neurones également (avec des améliorations)

Idée générale : la dérivée en un point donne l'accroissement de la fonction, si on fait un petit pas dans la direction opposée à celui-ci, on est certain de descendre un peu. On répète le processus jusqu'à arriver à un minimum local



DESCENTE DE GRADIENT

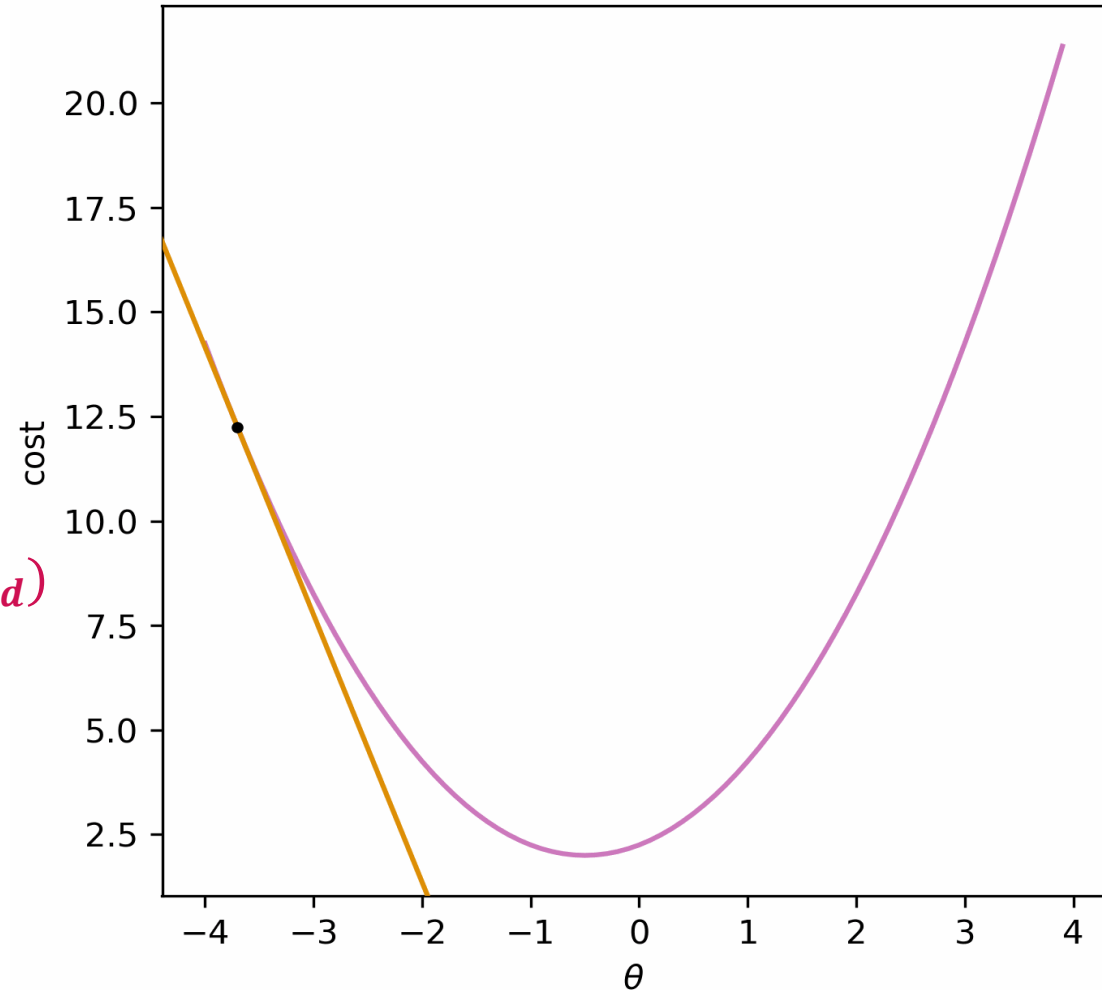
Formulation vectorielle :

- Initialisation de θ aléatoirement
- Mis à jour de θ :

$$\theta_{new} \leftarrow \theta_{old} - \alpha \frac{\partial}{\partial \theta_{old}} J(\theta_{old})$$

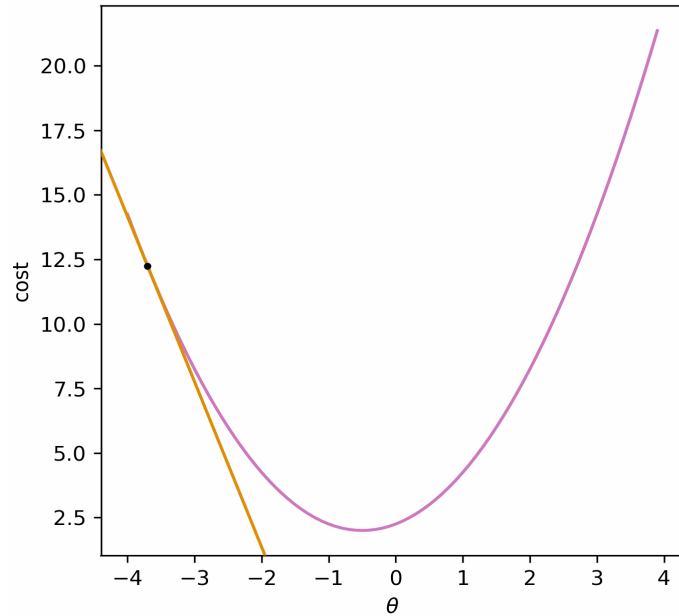
- Répéter jusqu'à convergence

$$|\theta_{new} - \theta_{old}| < \epsilon$$

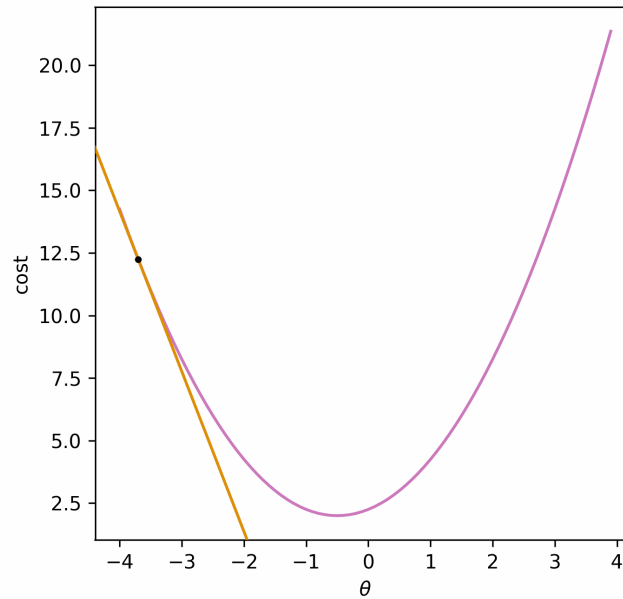


\ PAS D'APPRENTISSAGE (LEARNING RATE)

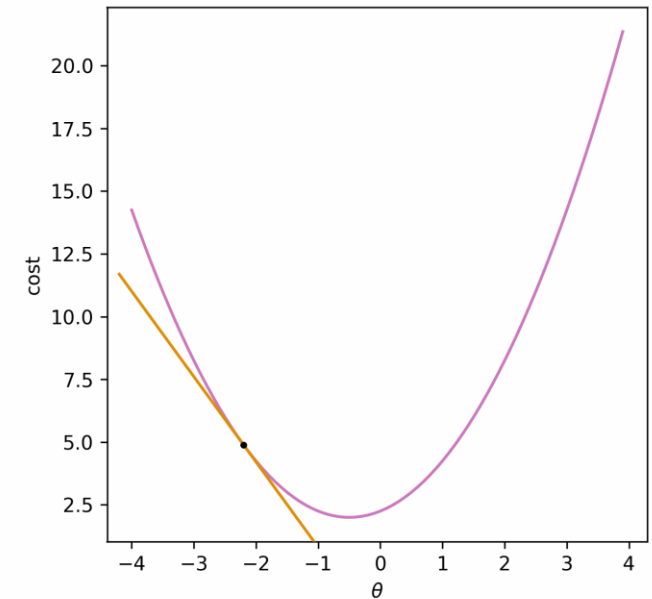
$$\theta_{new} \leftarrow \theta_{old} - \alpha \frac{\partial}{\partial \theta_{old}} J(\theta_{old})$$



α trop petit

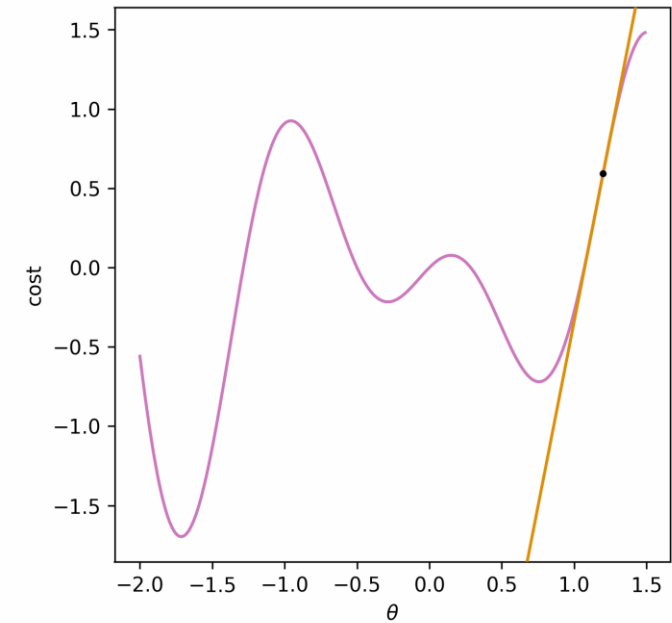
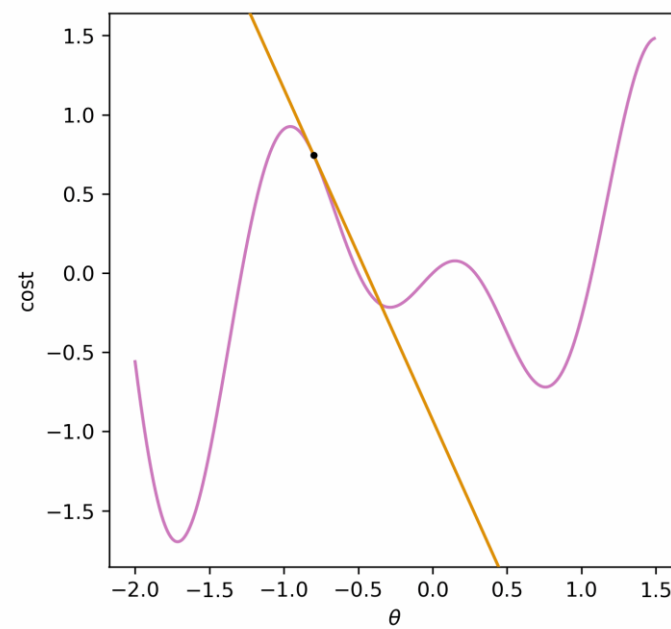
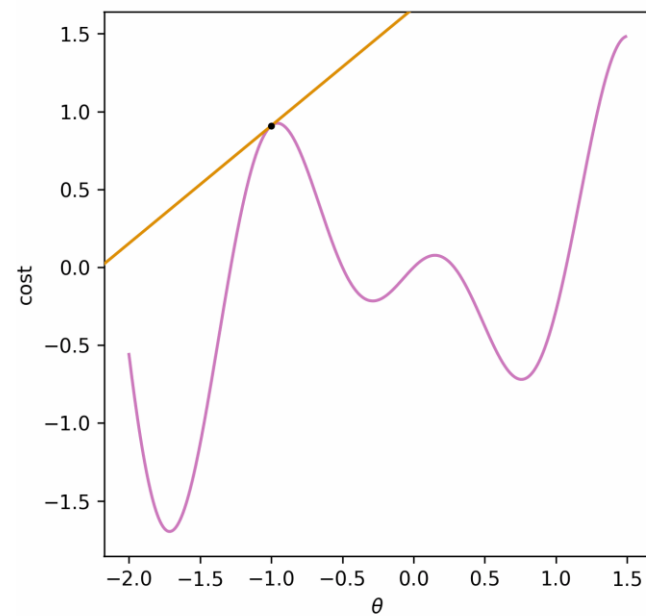


α correct



Vers l'infini et ...
(α trop grand)

LIMITE DE LA DESCENTE DE GRADIENT



VARIATIONS POUR "RÉDUIRE" CERTAINS PROBLÈMES

Problème des points selles :
Un classique en optimisation

Différentes variations sur la mise à jour :

- Adapter le learning rate à chaque poids
- Ajouter de l'inertie (momentum)
- Adapter le learning rate à l'inertie et au momentum

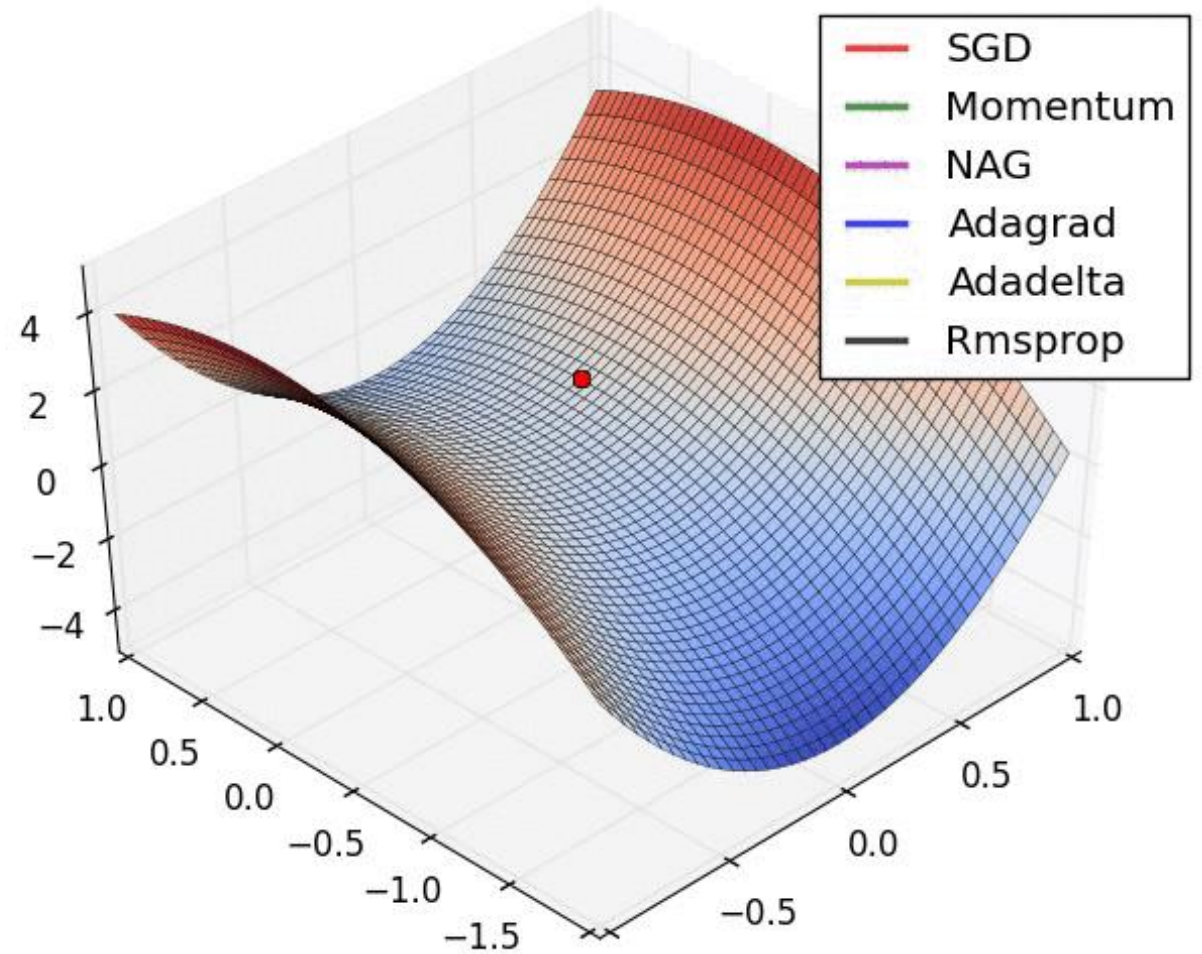


Image : <https://www.ruder.io/optimizing-gradient-descent/>

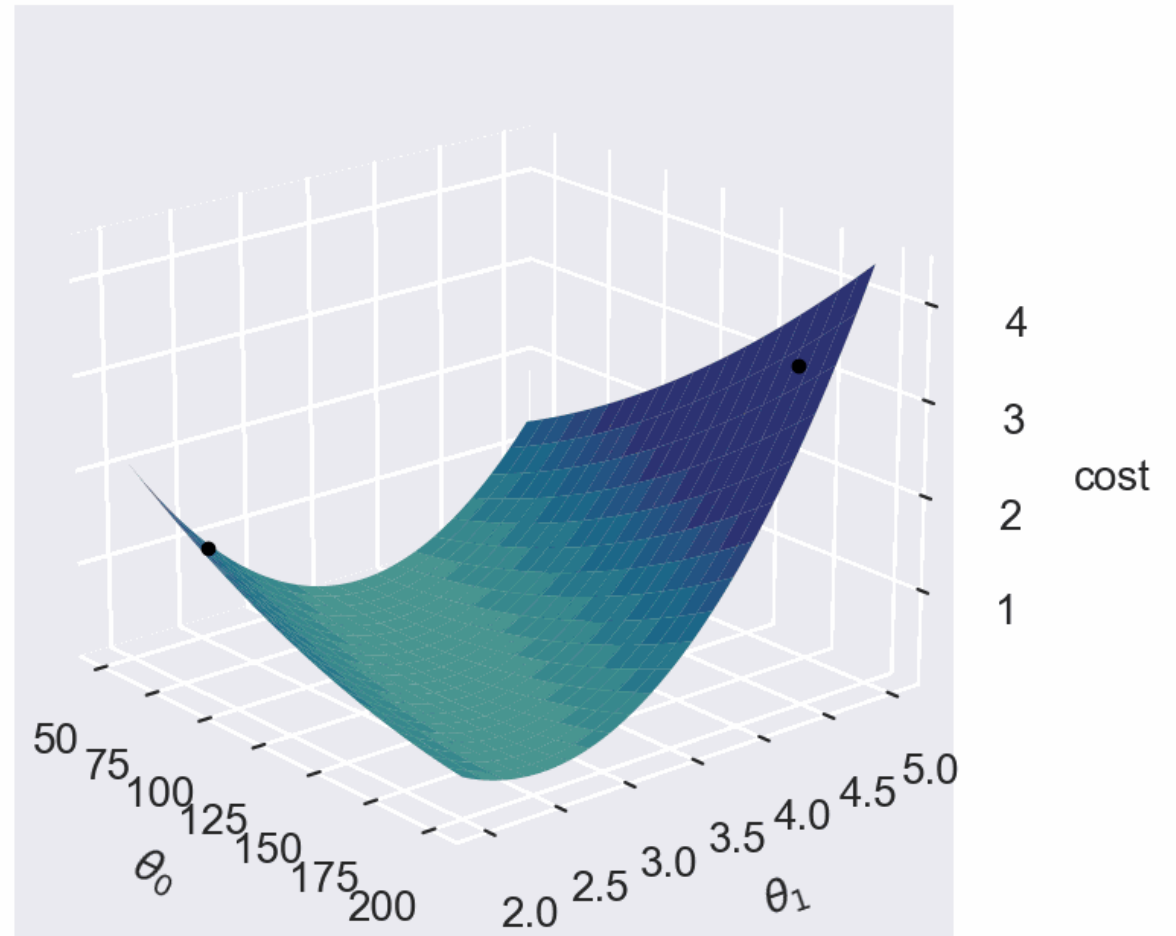
D'AUTRES PRÉCISIONS

Avec plus de caractéristiques θ est multidimensionnel, mais :

- La propriété de convexité est conservée

Le point d'initialisation va jouer :

- Initialisation aléatoire dans certains range pour éviter l'explosion.
(généralement, dans l'intervalle $[-1, 1]$)



ET LES DONNÉES DANS TOUT ÇA ?

Qu'est-ce que cela dit sur notre modèle ?

Qu'avec ces caractéristiques, notre modèle est optimal au sens des moindres carrés sur le **set d'entraînement** !

Est-ce que on peut en conclure quoi que ce soit sur la **généralisation** de notre modèle ?

Absolument pas, si les caractéristiques que l'on a ne permette pas de prédire correctement, un modèle « optimal » ne change rien

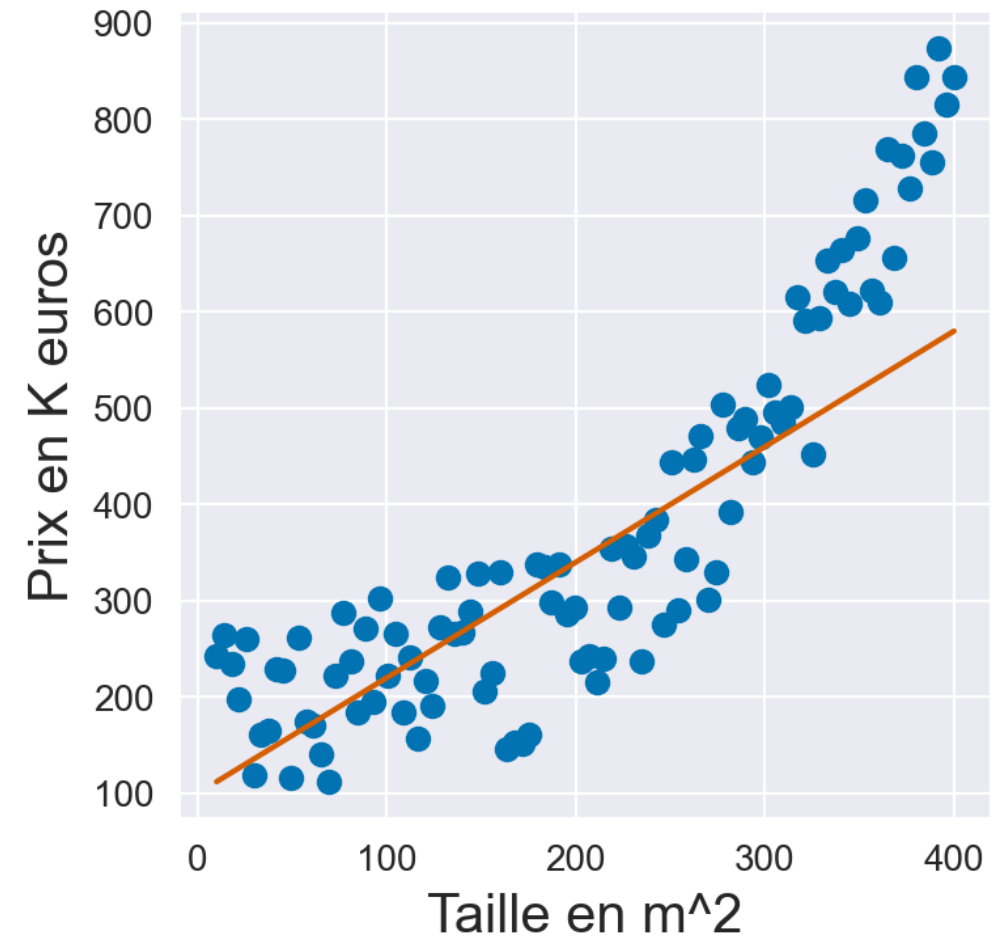
PROBLÈME POLYNOMIAL

Limite de la régression linéaire :

La vie n'est pas linéaire.

MAIS possibilités de jouer avec le jeu de données

Taille en m2	Taille en m2	Taille^2
25	25	625
100	100	10000
110	110	12100
175	175	30625
200	200	40000



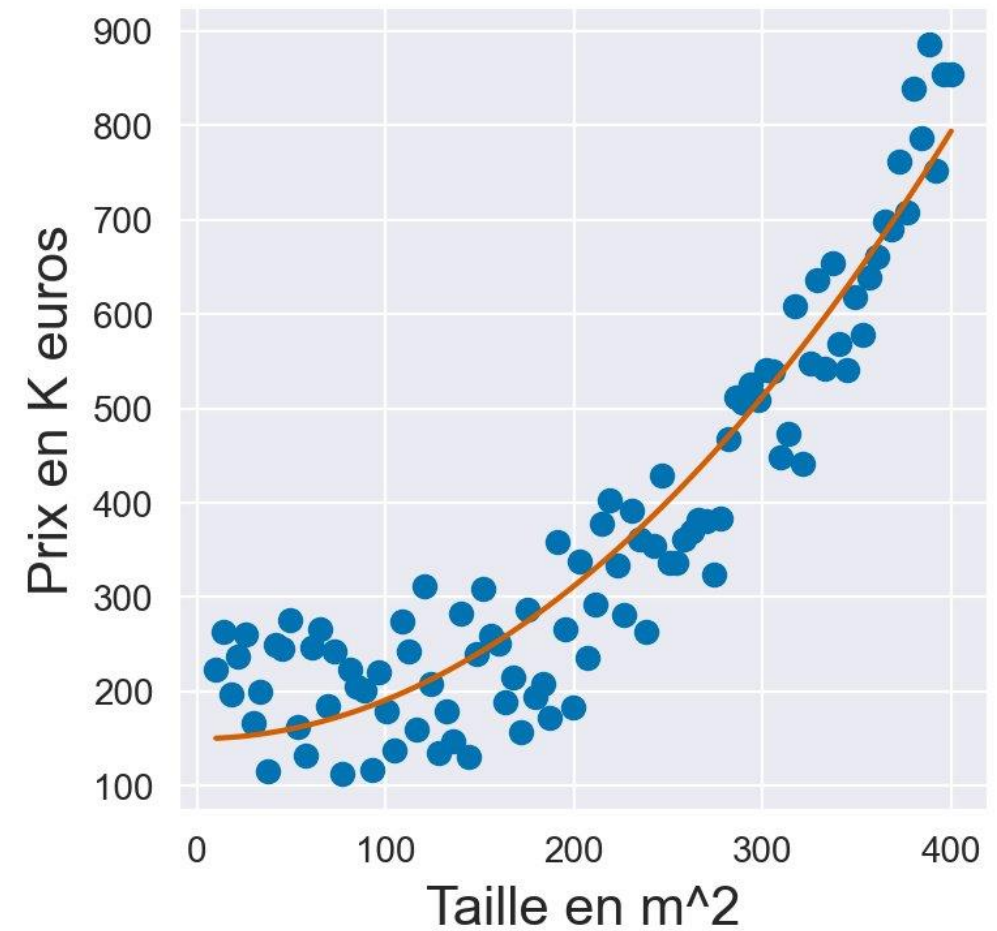
PROBLÈME POLYNOMIAL

Taille en m2
25
100
110
175
200

Taille en m2	Taille^2
25	625
100	10000
110	12100
175	30625
200	40000

La formulation devient :

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2$$



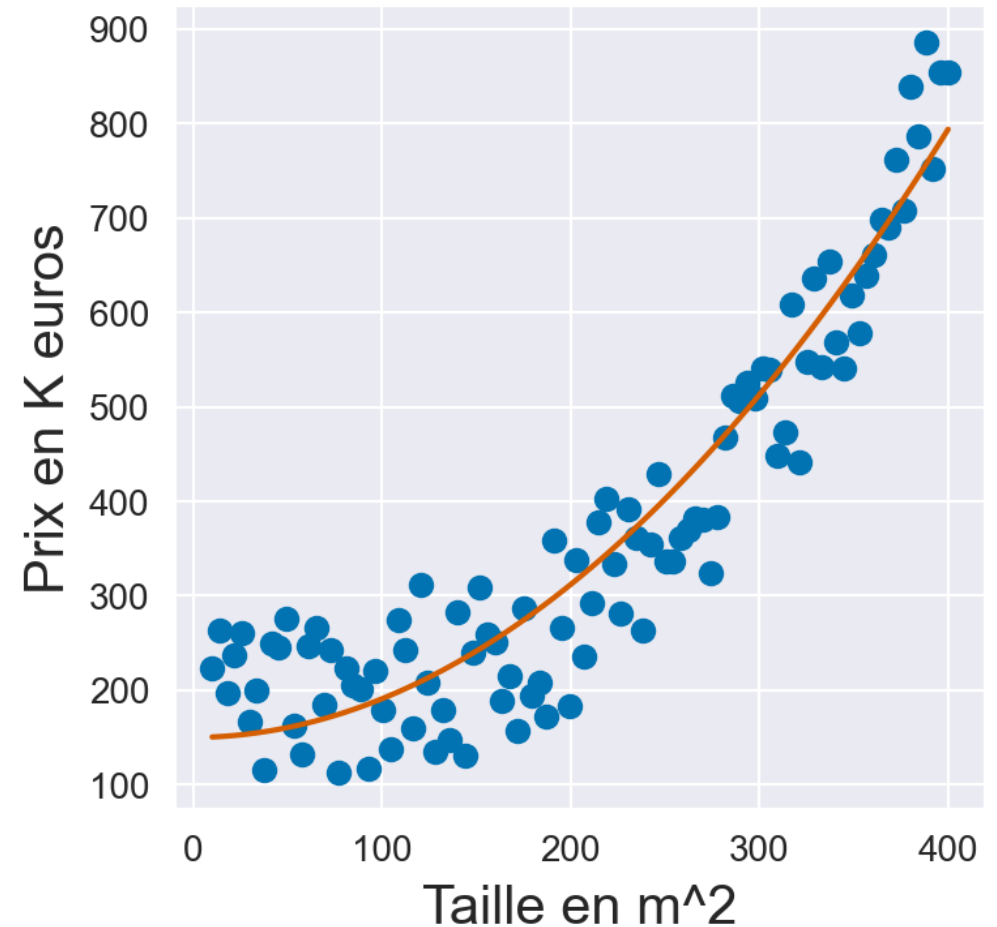
PROBLÈME POLYNOMIAL

Le modèle peut être optimisé avec la descente de gradient ET le problème est toujours convexe (le modèle est toujours linéaire par rapport aux entrées)

MAIS :

- Le nombre de caractéristiques est multiplié par le degré du polynome souhaité
- Si on a plusieurs caractéristiques, on doit les multiplier entre elles.

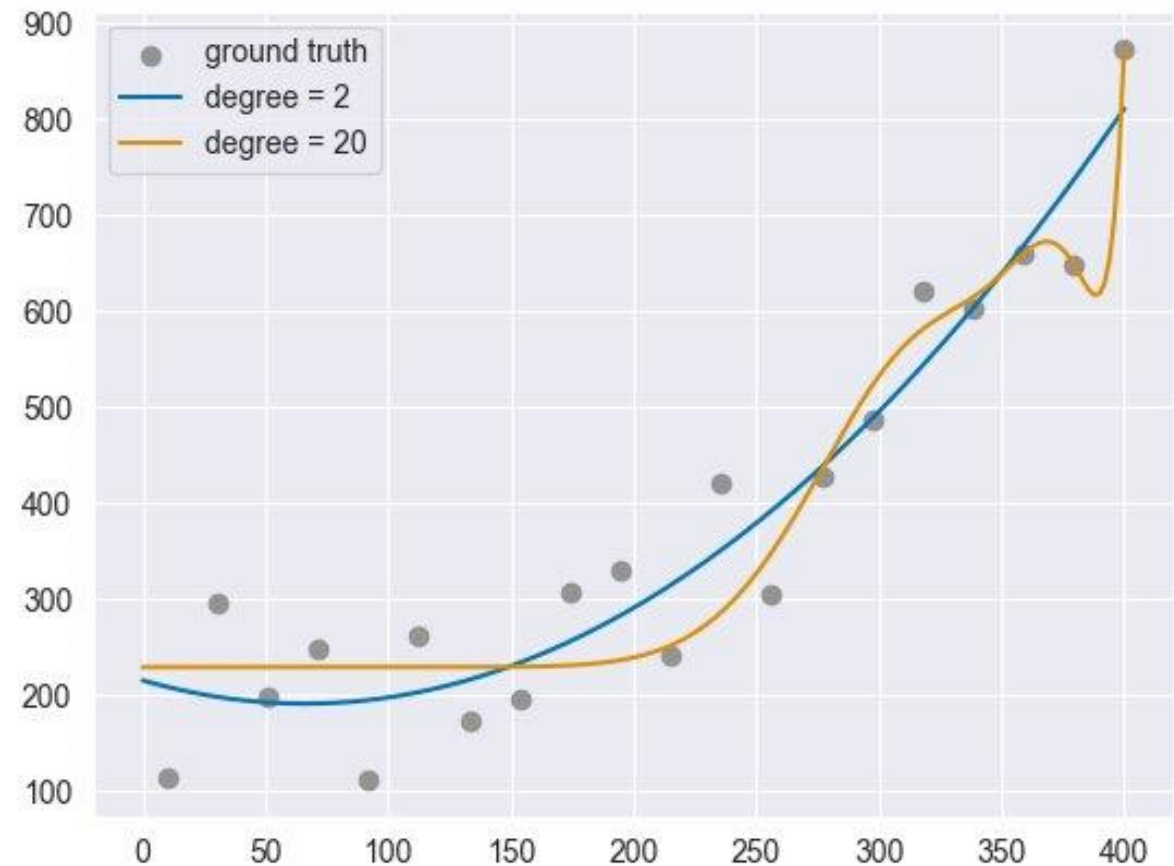
$$[x_0, x_1, x_2] \\ \rightarrow [x_0, x_1, x_2, x_0x_1, x_0x_2, x_1x_2, x_1x_2x_3]$$



LE SUR-APPRENTISSAGE

Quand il y a beaucoup de caractéristiques, le modèle risque de se focaliser sur des caractéristiques non-pertinentes (d'autant plus quand on aura des modèles avec des capacités expressives plus importantes)

Lequel des deux à l'air le plus raisonnable ?



\ AUTRE PROBLÈME QUI CAUSE LE SUR-APPRENTISSAGE

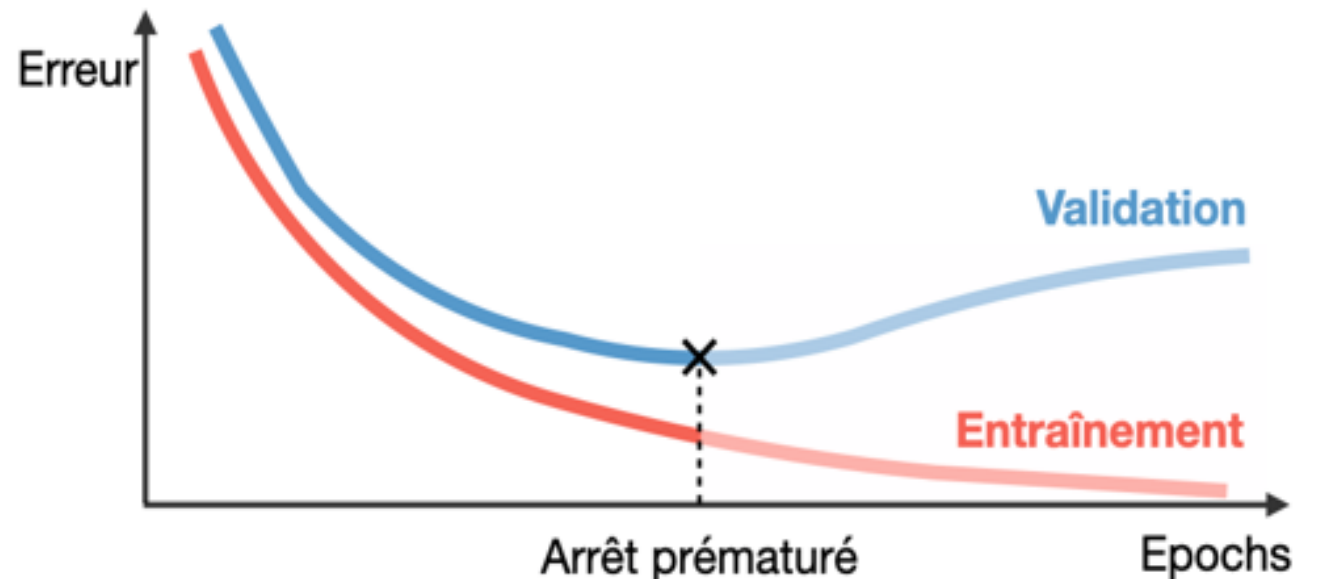
Taille en m2	Taille en m^2	rand1	rand2	...	Rand 99	Rand 100	Prix
25	25	12	23		-4	-10	275k
100	100	2	9		14	5	400k
110	110	3	2	...	8	-7	500k
175	175	5	17		-2	-10	700k
200	200	-4	5		7	-4	750k

X_1
 X_2

QUELLE FORME PREND LE SUR-APPRENTISSAGE ?

Une séparation entre la performance en entraînement et en test.

(Cela est vrai surtout avec des modèles de capacités plus importantes, comme les arbres de décisions ou les réseaux de neurones)



REGULARISATION

Régularisation : Technique permettant de limiter la capacité d'expression du modèle pour l'empêcher de sur-apprendre.

Que devient notre objectif maintenant ?

Poids de la
régularisation

Type de régularisation

$$\min_{\theta} \sum_{i=0}^n ||y^{(i)} - f_{\theta}(x^{(i)})||^2 + \boxed{\lambda} * \boxed{||\theta||^2}$$

En gros : On minimise les moindres carrés et on essaie de rapprocher les caractéristiques « inutiles » à 0.

REGULARISATION

L2, Weight decay, Ridge

$$\min_{\theta} \sum_{i=0}^n ||y^{(i)} - f_{\theta}(x^{(i)})||^2 + \lambda * ||\theta||^2$$

L1, Lasso

$$\min_{\theta} \sum_{i=0}^n ||y^{(i)} - f_{\theta}(x^{(i)})||^2 + \lambda * ||\theta||^1$$

Elastic Net

$$\min_{\theta} \sum_{i=0}^n ||y^{(i)} - f_{\theta}(x^{(i)})||^2 + \lambda * ||\theta||^2 + \beta * ||\theta||^1$$

Toutes des variations sur la même idée !

MIS À JOUR DU GRADIENT

Fonction objectif :

$$\min_{\theta} \sum_{i=0}^n ||y^{(i)} - f_{\theta}(x^{(i)})||^2 + \lambda * ||\theta||^2$$

Rappel de la règle de mis à jour :

$$\theta_{new} \leftarrow \theta_{old} - \alpha \frac{\partial}{\partial \theta_{old}} J(\theta_{old})$$

Mis à jour avec régularisation L2 :

$$\theta_{new} \leftarrow \theta_{old} - \alpha * [(X^T(X\theta_{old} - y) + 2\lambda\theta_{old})]$$

L'objectif reste un problème convexe !

CONCLUSION

- La descente de gradient est l'**algorithme central** du machine learning actuel.
- Permet d'obtenir une solution optimale au problème d'optimisation convexe
- Existe des astuces pour limiter les points selles, mais non-applicables pour les problèmes réellement non-convexes.
- La régularisation est là pour réduire la performance en entraînement et améliorer la performance en test.
- La régularisation ne remplace pas le travail sur les caractéristiques : Virer les caractéristiques inutiles, transformation polynomial, Standardisation des données.

